

Enterprise-Grade Autonomous Machine Learning Platform for Neurodegenerative Disease Protein Aggregation Prediction

Neurodegenerative pathologies, including Alzheimer's disease, Parkinson's disease, and Amyotrophic Lateral Sclerosis (ALS), are fundamentally driven by the misfolding and subsequent toxic aggregation of intrinsically disordered proteins (IDPs) such as Tau, Alpha-synuclein, TDP-43, Huntingtin, and FUS. Predicting the aggregation kinetics of these proteins is extraordinarily complex due to the influence of variable sequence lengths, environmental conditions, and dynamic post-translational modifications (PTMs)¹. Traditional computational modeling relies on molecular dynamics (MD) simulations that face severe timescale bottlenecks, making proteome-scale screening computationally prohibitive. The following architecture specifies an enterprise-grade, autonomous machine learning research platform designed to systematically predict protein aggregation across multiple neurodegenerative diseases. By orchestrating a closed-loop multi-agent system, this platform automatically generates hypotheses, writes deep learning code, audits execution safety, evaluates results against rigorous baselines, and iteratively refines its understanding of aggregation biophysics as new wet-lab data arrives.

1. Multi-Disease, Multi-Dataset Platform Architecture

To support a continuously expanding repertoire of neurodegenerative diseases, the platform must enforce strict data and algorithmic compartmentalization while simultaneously allowing for generalizable biophysical knowledge transfer.

Namespacing and Cross-Contamination Prevention

The foundational requirement of a multi-disease platform is preventing latent representations optimized for one protein's specific aggregation pathway (e.g., the NAC domain in Alpha-synuclein) from erroneously biasing predictions for another (e.g., the polyglutamine tract in Huntingtin). At the infrastructure level, this is achieved through strict tenant-based namespacing within the database and object storage layers. Algorithmically, cross-contamination is prevented by utilizing conditional embeddings. While a universal protein language model acts as the shared backbone, disease-specific projection heads are dynamically swapped based on the target. A conditioning vector representing the specific biological context is concatenated to the sequence embedding prior to the final multi-layer perceptron (MLP) layers, effectively partitioning the latent space and ensuring the network distinguishes the unique kinetic profiles of each protein family⁴.

Versioned Dataset Registry

Neurodegenerative datasets are highly mutable; new experimental fluorescence assays or mass spectrometry results are continuously integrated. The platform utilizes Data Version Control (DVC) backed by S3-compatible object storage to maintain an immutable, content-addressable history of all datasets⁵. Each modification to the dataset generates a unique SHA-256 commit hash. The experiment registry binds this specific data hash to the corresponding model weights and hyperparameters. This absolute traceability ensures that any historical experiment can be perfectly reproduced, preventing the silent degradation of model baselines due to unnoticed upstream data alterations⁵.

Unified Protein Aggregation Schema

Standardizing variable data across diverse diseases requires a flattened, schema-on-read tabular format, such as Apache Parquet, optimized for vectorized machine learning ingestion⁷. The schema must balance shared physical properties with highly specific experimental metadata.

Field Category	Column Name	Data Type	Description
Common Fields	unique_id	String (UUID)	Cryptographic identifier for the specific assay instance.
	sequence_string	String	The 1D amino acid text representation (variable length).
	protein_class	Categorical	Target IDP (e.g., Tau, Alpha-synuclein, FUS).
	concentration_mg_ml	Float	Molar concentration of the protein in the assay.
	temperature_c	Float	Experimental temperature conditions.
Target Labels	aggregation_severity	Ordinal Integer	Discretized aggregation bins (0:

			No, 1: Low, 2: Medium, 3: High).
	kinetic_rate	Float	Continuous rate of aggregation $\ln(v)$, if experimentally available.
Disease-Specific	metadata_json	JSON	Nested key-value pairs (e.g., chaperone presence, metal ions).
Modifications	ptm_array	Array[Int]	Positional encodings of Post-Translational Modifications.

Transfer Learning Strategy

Deep neural networks require massive datasets to learn the physical grammar of amino acids, yet disease-specific experimental datasets are often sparse (e.g., fewer than 100 highly curated rows). The architecture employs a tiered transfer learning protocol. The foundational model is first pre-trained on expansive, generalized aggregation repositories such as WALTZ-DB 2.0, which contains thousands of characterized amyloidogenic hexapeptides⁸, and CPAD 2.0 (Curated Protein Aggregation Database), which catalogs aggregation rate changes upon mutations¹⁰. Following this broad biophysical pre-training, the network undergoes parameter-efficient fine-tuning (PEFT), utilizing techniques such as Low-Rank Adaptation (LoRA), targeted strictly on the specific neurodegenerative target⁷. This strategy prevents rapid overfitting on the sparse disease dataset while preserving the generalized physical interactions learned from the broader databases.

2. Autonomous Experiment Execution Architecture

Self-Contained Experiment Script Generation

The autonomous cycle relies on an agentic Large Language Model (LLM) writing functional, isolated PyTorch scripts. To ensure these generated scripts execute reliably without human intervention, the agent operates within a strict Directed Acyclic Graph (DAG) generation pattern. The agent is not permitted to write arbitrary, unstructured code; instead, it is provided a rigid

object-oriented class template and instructed to override specific abstract methods, primarily focusing on network topology and optimizer configuration.

Experiment Harness Design

Every agent-generated script must inherit from a standardized ExperimentHarness base class. This contract ensures that the platform's deterministic execution node can interface uniformly with any proposed hypothesis, guaranteeing that critical steps such as cross-validation, metric computation, and artifact serialization are never omitted by a hallucinating agent.

Python

```
import torch
import torch.nn as nn
from abc import ABC, abstractmethod

class ExperimentHarness(ABC):
    def __init__(self, config_dict):
        self.config = config_dict
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.model = self.build_architecture().to(self.device)
        self.optimizer = self.configure_optimizers()

    @abstractmethod
    def build_architecture(self) -> nn.Module:
        """Agent must override this to define the neural network topology."""
        pass

    @abstractmethod
    def configure_optimizers(self) -> torch.optim.Optimizer:
        """Agent must override this to define the optimization strategy."""
        pass

    def train_evaluate_loop(self, train_loader, val_loader):
        """Locked method: Agent cannot override. Enforces strict evaluation."""
        # Implements standard PyTorch training loop with early stopping
        # Calculates Macro-F1 and Ordinal MAE
        # Serializes weights and logs to MLflow/Leaderboard
        pass
```

Output Structuring and Capture

Outputs generated within the isolated sandbox are rigorously structured for reliable ingestion into the leaderboard. The execution environment automatically wraps the `train_evaluate_loop` with tracking utilities (e.g., MLflow or Weights & Biases)⁶. The captured payload includes the final performance metrics (Macro-F1, per-class recall), the complete hyperparameter dictionary, serialized `.pt` model weights, and the standard output/error logs.

Failure Classification and Recovery Policies

In a continuous autonomous loop, execution failures are inevitable. The platform features an automated stack-trace parser to classify exceptions and trigger appropriate recovery behaviors:

Failure Type	Detection Mechanism	Recovery Policy	Retry Limit
Syntax / Compilation	SyntaxError, IndentationError	Traceback fed to the Coder agent for an immediate contextual rewrite.	3
Out of Memory (OOM)	RuntimeError: CUDA out of memory	Platform automatically halves the batch size or reduces hidden dimensions; resubmits without agent intervention.	3
Pathological Metrics	NaN loss or zero precision	Discards the current weights; triggers an Optuna Bayesian search to dynamically discover stable learning rates.	5 (Optuna trials)

Experiment Deduplication

To preserve expensive GPU compute, the platform must detect when a newly proposed experiment is functionally identical to a historical run. The system parses the agent-generated Python code into an Abstract Syntax Tree (AST) and generates a structural hash of the neural

network definition. It then calculates the cosine similarity between the proposed hyperparameter dictionary and those in the registry. If the structural hash matches and the hyperparameter similarity exceeds 0.98, the platform aborts the execution, logs the hypothesis as a duplicate, and prompts the Planner agent for a more divergent architectural approach¹³.

3. Leaderboard & Experiment Registry Design

Persistent Leaderboard Schema

Because datasets evolve as new proteins and labels are added, absolute metrics decay in comparative value. The leaderboard utilizes a relational schema structured around composite primary keys: Experiment_ID, Dataset_Version_Hash, and Disease_Target. To maintain a meaningful hierarchy, the leaderboard ranks models within the context of their specific dataset version, utilizing rolling windows to track relative algorithmic improvements across data epochs.

Reproducibility Metadata

True scientific autonomy requires cryptographic reproducibility. For every entry, the registry records an exhaustive metadata payload: the exact DVC hash of the dataset⁵, the global random seeds (including torch.backends.cudnn.deterministic = True), the complete feature inclusion list, the selected hyperparameters, and a serialized copy of the Docker container digest representing the exact software environment used during execution¹⁵.

Normalization Across Dataset Versions

As dataset sizes grow, raw accuracy or loss values shift unpredictably. To compare an experiment run on an older dataset version against a newer one, the platform utilizes baseline normalization strategies. Specifically, it employs a Logarithmic Overfitting Ratio (LOR) and Z-score scaling against a persistent "Dummy Baseline" (e.g., a standard Random Forest or a frozen 1D-CNN) that is automatically retrained on every new dataset version¹⁶. By evaluating the relative delta between the advanced deep learning model and the dummy baseline, the leaderboard maintains a normalized performance scale regardless of the absolute dataset difficulty.

Statistical Significance Testing

To ascertain whether a new model's performance gain is genuinely superior or merely the result of stochastic weight initialization noise, the platform mandates strict statistical testing. Every proposed architecture undergoes 5-Fold Cross-Validation on the 85% training split⁷. The Macro-F1 scores across the five folds are collected, and a paired Wilcoxon signed-rank test is

computed against the fold metrics of the reigning state-of-the-art (SOTA) model. A p -value < 0.05 is required to definitively declare the new hypothesis as superior and update the primary leaderboard champion.

4. Continual / Incremental Learning for Growing

Datasets

Continual Learning Strategies

Neurodegenerative research platforms experience a continuous influx of wet-lab assay data. Retraining massive transformer architectures from scratch upon the arrival of every new batch of peptide sequences is highly inefficient. The platform utilizes the Avalanche PyTorch framework, an end-to-end continual learning library designed specifically to manage non-stationary streams of data¹⁸.

Catastrophic Forgetting and Model Susceptibility

When incrementally fine-tuning, highly parameterized models like Transformers and deep CNNs are highly susceptible to catastrophic forgetting—overwriting the learned aggregation mechanics of older proteins to accommodate new patterns. To combat this, the Avalanche implementation utilizes a hybrid Replay Buffer and Regularization strategy²¹.

```
Python
from avalanche.training.supervised import Naive
from avalanche.training.plugins import ReplayPlugin, EWCPlugin

# Implement a Replay Buffer using reservoir sampling to retain historical diversity
replay_plugin = ReplayPlugin(mem_size=5000)

# Implement Elastic Weight Consolidation (EWC) to penalize changes to vital weights
ewc_plugin = EWCPlugin(ewc_lambda=0.4)

# Instantiate the Continual Learning Strategy
cl_strategy = Naive(
    model, optimizer, criterion,
    train_mb_size=32, train_epochs=5,
    plugins=[replay_plugin, ewc_plugin]
)
```

This dual approach ensures that the physical grammar learned from foundational datasets is mathematically protected via EWC penalties, while the replay buffer explicitly reminds the network of minority disease classes during the incremental update²².

Distribution Shift Detection

Incoming experimental data may feature novel amino acid mutations or entirely different concentration ranges, altering the underlying feature space. The platform autonomously monitors for distribution drift by executing two-sample Kolmogorov-Smirnov (K-S) tests on the continuous features (e.g., concentration) and Chi-Square tests on the categorical label distributions between the historical dataset and the new ingestion¹⁴. Significant statistical deviations automatically trigger a drift alert within the agent's state memory.

Triggering Retraining

Model retraining is governed by an event-driven orchestrator rather than a rigid cron schedule. Retraining is triggered if either:

1. **Data Volume Threshold:** The ingestion pipeline accumulates a predefined threshold of new sequences (e.g., 5% of the existing dataset size).
2. **Performance Degradation:** A newly ingested test batch reveals a drop in the production model's Macro-F1 score below acceptable thresholds, indicating that the model's current latent space cannot accurately map the newly discovered biological phenomena.

5. Variable-Length Sequence Modeling

Architectures for Variable Sequences

Peptide sequences range from short amylogenic hexapeptides (6-mers) to full-length intrinsically disordered domains exceeding 100 amino acids. Fixed-window models, such as basic Fully Connected Networks (FCNs), crash upon encountering variable length inputs. The platform utilizes Transformer architectures, specifically Protein Language Models (pLMs) like Meta AI's ESM-2 (Evolutionary Scale Modeling)²⁴. Transformers rely on self-attention mechanisms that compute pairwise interactions across the entire chain without requiring fixed spatial dimensions, making them natively capable of ingesting variable-length strings⁷.

Fixed-Size Representation Extraction

To route the variable-length sequence outputs from the ESM-2 transformer into the downstream classification MLP, the architecture must extract a fixed-size 1D representation. This is primarily achieved through Mean-Pooling the hidden states across the sequence length dimension, creating a unified global embedding that encapsulates the overall biophysical propensity of the entire peptide. Alternatively, extracting the embedding of the Beginning-of-Sequence (<cls>) token provides an aggregated summary state optimized during pre-training²⁴.

Padding and Masking Strategies

For efficient GPU utilization, variable-length sequences must be batched together as rectangular tensors. Sequences are padded with dummy tokens (e.g., <PAD>) to match the longest sequence in the batch. Crucially, strict boolean Attention Masks are passed simultaneously to the transformer layers. These masks (1 for valid amino acids, 0 for padding) explicitly instruct the self-attention heads to mathematically ignore the padded regions, ensuring that arbitrary zeros do not corrupt the calculation of biological interaction forces⁷.

When to Use Sliding Windows

While global embeddings are ideal for short-to-medium peptides, sliding windows remain mathematically appropriate for full-length proteins (e.g., full-length Tau). By passing overlapping 20-amino-acid fragments through the network, the model can precisely pinpoint localized "hotspots"—the exact spatial origin of the aggregation-prone beta-sheet—rather than diluting the signal across a massive global sequence average⁷.

6. Multi-Agent Orchestration for Long-Running Research Platforms

Framework Comparison

Sustaining a multi-month autonomous research lifecycle requires stateful workflow orchestration. Basic linear chaining frameworks are inadequate because they cannot easily support iterative debugging, infinite reflection loops, or state checkpointing¹³. The architecture mandates the use of LangGraph, which models the multi-agent system as a state machine operating on a Directed Acyclic Graph (DAG) with cyclic conditional routing²⁷.

Role Decomposition and Handoff Contracts

The multi-agent system divides the cognitive load into distinct, specialized persona nodes that communicate exclusively via a shared, typed AgentState dictionary.

Agent Role	Function	Handoff Contract / Interface
Planner	Formulates novel ML hypotheses based on literature and historical leaderboard metrics.	Outputs hyperparameter grids and architectural text descriptions to the Coder.
Coder	Translates the Planner's hypothesis into a PyTorch script adhering to the ExperimentHarness.	Outputs a raw Python string to the Auditor.
Auditor	Performs static code analysis via AST parsing to detect security flaws or syntax errors.	If safe, passes code to Executor. If flawed, routes feedback back to the Coder.

Executor	(Non-LLM) Deterministically pushes code to the gVisor sandbox and runs the PyTorch training loop.	Outputs captured logs, metrics, and stack traces to the Evaluator.
Evaluator	Analyzes the training curves and metrics; determines if the hypothesis succeeded or failed.	Appends structured critiques to the critique_history state array, routing back to the Planner ¹³ .

Persistent Memory and State Checkpointing

Because the research loop runs asynchronously and indefinitely, volatile memory is insufficient. LangGraph utilizes external checkpoint savers (e.g., PostgreSQL or SQLite) to persist the AgentState graph at every node transition²⁷. If the orchestration server is interrupted or a new dataset triggers an asynchronous retraining event, the agent resumes seamlessly from its exact previous state. Long-term semantic memory is managed via a vector database (e.g., FAISS), allowing the Planner to retrieve the conceptual history of experiments across months of iterations.

Hypothesis Novelty Gating

In highly automated loops, agents frequently fall into repetitive optimization cycles, re-proposing nearly identical variations of failed models. A strict novelty gate is implemented before the Planner passes a hypothesis to the Coder. The proposed hypothesis is converted into a dense vector embedding and queried against the FAISS index of historical experiments. If the semantic similarity exceeds a predefined threshold, the hypothesis is blocked, and the Planner is forced to increase its generation temperature and explore a divergent architectural path.

7. Typed Tool Registry Design

Schema Contracts for Agent Tools

Agent nodes interface with the external infrastructure through a registry of strictly typed tools. By utilizing Pydantic models, the platform enforces schema validation, ensuring the LLM does not hallucinate invalid arguments. Core tools include:

- propose_experiment(architecture: str, learning_rate: float, dropout: float)
- fetch_literature(query: str, max_results: int)
- detect_drift(historical_dvc_hash: str, new_dvc_hash: str)
- audit_code(python_string: str)

Tool versioning is enforced via API routing. As the platform evolves, older tools are maintained in the registry but flagged as deprecated, ensuring that an agent attempting to reproduce a six-month-old experiment can still access the exact historical tooling endpoints.

Static Code Auditing

Permitting an LLM to generate and execute Python code introduces profound remote code execution (RCE) vulnerabilities²⁹. Prior to execution, the Auditor node runs the script through RestrictedPython. This library intercepts compilation, rewriting the Abstract Syntax Tree (AST) to route all attribute lookups, module imports, and loop iterations through a highly curated allowlist (safe_builtins)³¹. Furthermore, pattern detectors like sandboxaudit scan the AST for known CVE escape techniques, actively blocking dangerous calls such as eval(), exec(), or metaclass manipulation²⁹.

Python

```
from RestrictedPython import compile_restricted, safe_builtins
```

```
def execute_agent_code(agent_code_string):
    try:
        # Compiles to a restricted AST, preventing unauthorized OS or network calls
        byte_code = compile_restricted(agent_code_string, '<inline>', 'exec')
        restricted_globals = {'__builtins__': safe_builtins}
        exec(byte_code, restricted_globals, {})
    except SyntaxError as e:
        return f"Auditor rejected code: {str(e)}"
```

Safe Sandboxing on Shared Compute

Even with AST rewriting, the generated PyTorch script must execute in strict hardware isolation to prevent GPU memory starvation or credential theft on shared IIT Gandhinagar computing clusters. Traditional Docker containers share the host Linux kernel, which is an insufficient security boundary for untrusted code³⁶. The platform utilizes gVisor (a user-space kernel implementation) or Kata Containers (hardware-enforced microVMs) to sandbox the execution. gVisor intercepts all system calls from the agent's container, neutralizing host-kernel exploits and providing a robust, software-enforced boundary for compute-heavy ML workloads³⁶.

8. Protein Embedding & Feature Toolchain

Protein Language Model Checkpoints

The architecture integrates pre-trained ESM-2 checkpoints to convert sequence text into information-rich mathematical vectors. For rapid, iterative hyperparameter searches driven by

Optuna, the platform utilizes the lightweight esm2_t12_35M_UR50D (35 million parameters). For finalizing predictions on the Holdout Vault, the system scales to esm2_t33_650M_UR50D (650 million parameters), which captures deep evolutionary grammar, contact maps, and secondary structure folding propensities without the massive computational overhead of explicit molecular dynamics⁷.

Aggregation-Specific Feature Integration

While ESM-2 models evolutionary viability, it is not explicitly trained on aggregation physics. The platform algorithmically calls an ensemble of domain-specific predictors to generate a supplementary biophysical feature matrix:

- **TANGO:** Calculates cross-beta sheet aggregation propensity based on statistical thermodynamics³⁹.
- **Waltz:** Specifically identifies amylogenic regions distinct from amorphous aggregates⁹.
- **CamSol:** Computes intrinsic solubility profiles, highlighting hydrophobic exposure at specific pH levels⁴¹.
- **Zygggregator:** Calculates position-specific amyloid formation scores⁴³.

These classical biophysical outputs are concatenated directly to the high-dimensional ESM-2 embedding tensor, ensuring the downstream MLP leverages both deep semantic grammar and rigid thermodynamic physics⁷.

Encoding Post-Translational Modifications (PTMs)

Standard language models treat amino acids as static letters, but PTMs (such as lysine acetylation) fundamentally alter a protein's electrical charge and aggregation behavior. Because ESM-2 was trained on natural sequences, it treats an artificial 'X' (representing an acetylated lysine) as an unknown token⁷. To resolve this, the architecture employs a dual-stream encoding mechanism:

1. The sequence is mapped with 'X' reverted to its natural state ('K') and passed through ESM-2 to extract the base structural embedding.
2. A localized binary masking array (the "PTM Map") is generated, indicating the exact spatial coordinates of the chemical modifications.
3. These streams are fused, allowing the network to modulate the base structural embedding precisely where the electrical charge has been neutralized by the acetylation⁷.

Multi-Modal Drug Co-Discovery

As the platform extends beyond predicting aggregation toward predicting therapeutic interventions, it integrates the ConPLex (Contrastive Protein-Ligand Binding) architecture. ConPLex utilizes a two-tower network: Tower 1 embeds the protein sequence using the pLM, while Tower 2 embeds a small molecule drug candidate (via a Graph Neural Network operating on SMILES strings). The model maps both entities into a shared contrastive latent space, allowing the platform to rapidly predict if a specific drug compound will bind to and neutralize the misfolded IDP¹.

9. Imbalanced Multi-Class Classification Across Varying Dataset Sizes

Handling Severe Class Imbalance

Neurodegenerative wet-lab data is intrinsically imbalanced; severe "High" aggregation events are vastly outnumbered by stable, non-aggregating variants⁷. Standard Cross-Entropy loss functions fail in this regime, as the network achieves artificially high accuracy by blindly predicting the majority class, completely ignoring the critical toxic variants. While resampling techniques (like SMOTE) distort the biological reality of the dataset, the platform utilizes **Focal Loss**. Focal loss dynamically scales the gradient based on prediction confidence, adding a modulating factor $(1 - p_t)^\gamma$ to the standard cross-entropy calculation⁴⁷.

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t)$$

This formulation aggressively down-weights the loss generated by easily classified stable proteins, forcing the optimizer to focus its learning capacity entirely on the sparse, hard-to-classify toxic mutations⁴⁷.

```
Python
import torch
import torch.nn as nn
import torch.nn.functional as F

class MultiClassFocalLoss(nn.Module):
    def __init__(self, gamma=2.0, alpha=None):
        super(MultiClassFocalLoss, self).__init__()
        self.gamma = gamma
        self.alpha = alpha # Class balancing weights

    def forward(self, inputs, targets):
        log_pt = F.log_softmax(inputs, dim=1)
        pt = torch.exp(log_pt)
        # Apply the focal modulation factor
        focal_term = (1 - pt) ** self.gamma
        loss = F.nll_loss(focal_term * log_pt, targets, weight=self.alpha)
```

return loss

Ordinal Regression Architectures

The target labels (No, Low, Medium, High) represent a strict chronological and severity-based kinetic progression. Treating them as independent categories via standard multi-class classification is mathematically flawed, as it penalizes a "Medium" prediction for a "High" target exactly the same as a "No" prediction⁷. The architecture reframes the task as Ordinal Regression using frameworks such as CORAL (Consistent Rank Logits) or CORN (Conditional Ordinal Regression for Neural Networks). The final layer is partitioned into $K - 1$ sequential binary classifiers that share a weight constraint⁵⁰. This imposes a rank-consistent loss penalty, heavily punishing the neural network for extreme prediction errors (e.g., predicting "No" when the ground truth is "High") and significantly improving predictive balance across the severity spectrum⁷.

Robust Evaluation Metrics

Because the data size fluctuates and is highly imbalanced, global accuracy is discarded as a primary metric. The platform's evaluation oracle strictly relies on the **Macro-F1 Score**, which treats all classes equally regardless of support size, ensuring the model performs well on minority toxic classes. Additionally, **Mean Absolute Error (MAE)** is tracked to quantify the magnitude of deviation along the ordinal severity scale, keeping leaderboard comparisons stable even as the underlying N of the dataset grows⁷.

10. Literature Integration for Autonomous Research

API Integration and RAG Architecture

To ensure the Planner agent bases its architectural hypotheses on contemporary biophysics rather than hallucinated mechanisms, the platform integrates automated literature retrieval. Python scripts utilize the NCBI E-utilities API to fetch newly published PubMed and bioRxiv abstracts matching dynamic keywords (e.g., "Tau" AND "aggregation kinetics" AND "machine learning")⁷.

Extracting Knowledge via RAG

Because scientific literature vastly exceeds LLM context windows, the platform implements a Retrieval-Augmented Generation (RAG) architecture. Downloaded abstracts are parsed and chunked using a semantic text splitter, ensuring that complete biochemical paragraphs are not truncated mid-sentence. These chunks are embedded using a domain-specific model (such as PubMedBERT) and stored in a local FAISS or Chroma vector database. When the Planner agent formulates a new hypothesis, it executes a similarity search against the vector database. The top retrieved chunks are injected directly into the LLM's system prompt. This context injection allows the agent to discover 2026 breakthroughs—such as the impact of acetylation on

electrical charge—and explicitly map those concepts into its next PyTorch architecture design⁷.

11. Dataset Quality & Label Consistency Across Disease Expansion

Unified Label Ontology and Validation

As the platform expands to cover TDP-43, Huntingtin, and other IDPs, maintaining a unified label ontology is critical. The platform employs Pydantic models as a rigid programmatic validation layer during data ingestion. Incoming CSVs are verified to ensure that sequence strings contain only valid amino acid characters, that concentration variables are positive floats, and that raw RFU fluorescence readings align with the standardized (0, 1, 2, 3) ordinal tier mapping⁷.

Handling Conflicting Labels

Biologists frequently annotate identical peptide sequences differently depending on subtle assay variations. When conflicting labels for the same sequence collide in the database, the pipeline algorithmically isolates the environmental variables (such as pH or concentration). If the conditions are identical, the system resolves the conflict by averaging the continuous kinetic scores before discretizing them into the final ordinal bin, preventing contradictory ground-truth labels from confusing the neural network during gradient descent⁷.

Inter-Annotator Agreement Tracking

As multiple researchers contribute to the platform's datasets (such as CPAD 2.0), the platform continuously monitors labeling consistency. The system utilizes Python's statsmodels library to calculate **Fleiss' Kappa** (for multiple annotators) and sklearn's **Cohen's Kappa** (for two annotators). By actively tracking this inter-annotator agreement score, the platform ensures that the underlying training data remains highly consistent; if the Kappa score drops below a reliable threshold (e.g., < 0.60), the system alerts the laboratory directors to standardize their wet-lab annotation protocols⁵⁴.

12. Evaluation Protocol Design

Preventing Sequence Data Leakage

In protein bioinformatics, random train/test splitting introduces a fatal flaw known as data leakage. If a dataset utilizes sliding windows or single-point mutations (e.g., A53T in Alpha-synuclein), highly homologous sequences will appear in both the training and testing sets. The neural network will bypass learning generalized physics and instead memorize the localized sequence motifs, yielding artificially inflated leaderboard scores⁷.

Peptide-Level and Cluster-Based Splitting

The evaluation protocol strictly enforces **Peptide-Level Splitting**. The platform utilizes

clustering algorithms (such as MMseqs2) to group homologous sequences based on a predefined similarity threshold (e.g., 50% sequence identity). The algorithm ensures that all mutational variants derived from a single parent cluster are confined entirely to either the training set or the Holdout Vault test set. This strict stratification forces the AI to generalize its biophysical understanding to completely unseen protein topographies.

The Locked Evaluation Oracle

To guarantee that the autonomous agent is genuinely discovering universal biophysical laws rather than gaming the system through aggressive hyperparameter optimization, the architecture implements a mathematically inviolable "Holdout Vault"⁷. During the initial DVC data ingestion, 15% of the clustered dataset is permanently sequestered. The agentic loop is restricted to executing its Bayesian searches and Cross-Validation strictly on the remaining 85%. The vault is accessed exclusively by a locked Oracle execution node at the very end of the training cycle. The Oracle evaluates the model against the unseen data and outputs the definitive Macro-F1 score to the leaderboard, ensuring absolute integrity and scientific reproducibility for the enterprise platform².

Works cited

1. Contrastive learning in protein language space predicts interactions between drugs and protein targets | PNAS, <https://www.pnas.org/doi/10.1073/pnas.2220778120>
2. Default Machine Learning Hyperparameters Do Not Provide Informative Initialization for Bayesian Optimization - arXiv, <https://arxiv.org/html/2602.08774v1>
3. Galactic Geochelone changelog — ROS 2 Documentation, <https://docs.ros.org/en/rolling/Releases/Galactic-Geochelone-Complete-Changelog.html>
4. Learning to Align Molecules and Proteins: A Geometry-Aware Approach to Binding Affinity, <https://arxiv.org/html/2509.20693v1>
5. Data Versioning for ML: Keep Experiments Reproducible Across Teams - Transcloud, <https://wetranscloud.com/blog/data-versioning-for-ml-reproducible-experiments-teams/>
6. Model Versioning and Tracking in Machine Learning | by Timothy Kimutai - Medium, <https://timkimutai.medium.com/model-versioning-and-tracking-in-machine-learning-102d75b07b8b>
7. Drug Discovery, uploaded:Drug Discovery
8. waltz-db 2.0 - FAIRsharing, <https://fairsharing.org/FAIRsharing.EGn1ut>
9. WALTZ-DB: a benchmark database of amyloidogenic hexapeptides - Oxford Academic, <https://academic.oup.com/bioinformatics/article/31/10/1698/177770>
10. (PDF) CPAD, Curated Protein Aggregation Database: A Repository of Manually Curated Experimental Data on Protein and Peptide Aggregation - ResearchGate, https://www.researchgate.net/publication/299650004_CPAD_Curated_Protein_Ag

gregation Database A Repository of Manually Curated Experimental Data on Protein and Peptide Aggregation

11. CPAD 2.0: a repository of curated experimental data on aggregating proteins and peptides, <https://pubmed.ncbi.nlm.nih.gov/31979981/>
12. peft/examples/lora_finetuning_transformer_engine/README.md at main - GitHub, https://github.com/huggingface/peft/blob/main/examples/lora_finetuning_transformer_engine/README.md
13. LangGraph Tutorial: Self-Correcting AI Agents and Agent Loops | ActiveWizards, <https://activewizards.com/blog/a-deep-dive-into-langgraph-for-self-correcting-ai-agents/>
14. Detecting infinite loops in LangGraph multi-agent systems (before tokens explode) - Reddit, https://www.reddit.com/r/LangChain/comments/1r2mdz1/detecting_infinite_loops_in_langgraph_multiagent/
15. Isolate AI code execution with Agent Sandbox | GKE AI/ML - Google Cloud Documentation, <https://docs.cloud.google.com/kubernetes-engine/docs/how-to/agent-sandbox>
16. Numerical data: Normalization | Machine Learning - Google for Developers, <https://developers.google.com/machine-learning/crash-course/numerical-data/normalization>
17. Best Practices for Machine Learning Experimentation in Scientific Applications - arXiv, <https://arxiv.org/html/2511.21354v1>
18. Avalanche RL: an End-to-End Library for Continual Reinforcement Learning - GitHub, <https://github.com/continualai/avalanche-rl>
19. Avalanche: and End-to-End Library for Continual Learning based on PyTorch - Medium, <https://medium.com/pytorch/avalanche-and-end-to-end-library-for-continual-learning-based-on-pytorch-a99cf5661a0d>
20. Avalanche: An End-to-End Library for Continual Learning - CVF Open Access, https://openaccess.thecvf.com/content/CVPR2021W/CLVision/papers/Lomonaco_Avalanche_An_End-to-End_Library_for_Continual_Learning_CVPRW_2021_paper.pdf
21. Current Release | Avalanche - ContinualAI, <https://avalanche.continualai.org/getting-started/alpha-version>
22. Avalanche: A PyTorch Library for Deep Continual Learning - IRIS, <https://iris.luiss.it/retrieve/82017e56-ff2b-4741-9148-7bda4d23dc47/github.com/continualAI/continual-learning-baselines.pdf>
23. Antonio Carta - Continual Causality, <https://www.continualcausality.com/assets/materials/Carta.pdf>
24. ESM-2 150M - BioLM, <https://biolm.ai/models/esm2-150m/>
25. Protein Language Models Explained for Bench Scientists - Orbion, <https://www.orbion.life/blog/protein-language-models-explained-for-bench-scientists>
26. ESM - Hugging Face, https://huggingface.co/docs/transformers/model_doc/esm
27. Building Your Own AI Agent: A Practical Guide with LangGraph - DEV Community,

- <https://dev.to/midas126/building-your-own-ai-agent-a-practical-guide-with-langgraph-h-4no>
28. Human-in-the-Loop AI Agents in LangGraph: The 2026 Production-Ready Approach, <https://growwstacks.com/blog/human-in-the-loop-ai-agents-langgraph>
 29. AI-Powered Security Audit Report - 10 Vulnerabilities Found (Code Injection, SSRF, API Security) · Issue #3939 · browser-use/browser-use - GitHub, <https://github.com/browser-use/browser-use/issues/3939>
 30. Securing Python Code Execution: How We Protected Our Server from Untrusted Code, <https://blog.mornati.net/securing-python-code-execution-how-we-protected-our-server-from-untrusted-code>
 31. AnswerDotAI/safepyrun: Safe(ish) running of python code - GitHub, <https://github.com/AnswerDotAI/safepyrun>
 32. API overview - RestrictedPython 8.2 documentation, <https://restrictedpython.readthedocs.io/en/latest/usage/api.html>
 33. The idea behind RestrictedPython - Read the Docs, <https://restrictedpython.readthedocs.io/en/latest/idea.html>
 34. kriskimmerle/sandboxaudit: AI Agent Sandbox Security Auditor - Runtime probe for agent execution environment isolation (OWASP Agentic Top 10) · GitHub, <https://github.com/kriskimmerle/sandboxaudit>
 35. Python eval() Function Detection Vulnerability - Sourcery AI, <https://www.sourcery.ai/vulnerabilities/python-lang-security-audit-eval-detected>
 36. How to sandbox AI agents in 2026: MicroVMs, gVisor & isolation strategies | Blog, <https://northflank.com/blog/how-to-sandbox-ai-agents>
 37. How Every Major Tech Company Is Sandboxing AI Agents Differently - Medium, <https://medium.com/@earlperry562/how-every-major-tech-company-is-sandboxing-ai-agents-differently-f41b65f14d8a>
 38. Remote Execution by Design: Sandboxing AI Agents with gVisor & Pipelock - FOSS United Foundation, <https://fossunited.org/c/indiafoss/2026/cfp/d6mltdb2aq>
 39. Predicting Aggregation Region in Proteins with Machine Learning, Based on Tertiary Structure Magre-II - SciELO, <https://www.scielo.br/j/jbchs/a/ryCkb3skjMZGNdy3dzv9Whd/?lang=en>
 40. Software | VIB Switch Laboratory, <https://switchlab.org/software/>
 41. BioLMSol | BioLM, <https://biolm.ai/models/biolmsol/>
 42. Sequence-based prediction of the intrinsic solubility of peptides containing non-natural amino acids - University of Cambridge, <https://api.repository.cam.ac.uk/server/api/core/bitstreams/d96911c3-3ce5-49b4-9148-786c89df59ad/content>
 43. AGGRESKAN: Method, application, and perspectives for drug design | Request PDF - ResearchGate, https://www.researchgate.net/publication/51901834_AGGRESKAN_Method_application_and_perspectives_for_drug_design
 44. An efficient machine-learning framework for predicting protein post-translational modification sites - PMC, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12379237/>
 45. Learning a PRECISE language for small-molecule binding - bioRxiv,

- <https://www.biorxiv.org/content/10.64898/2026.01.04.697581v1.full-text>
46. (PDF) Contrasting drugs from decoys - ResearchGate, https://www.researchgate.net/publication/365167800_Contrasting_drugs_from_decoys
 47. FocalLoss.pytorch/Explanation.md at master · ashawkey/FocalLoss.pytorch · GitHub, <https://github.com/ashawkey/FocalLoss.pytorch/blob/master/Explanation.md>
 48. itakurah/focal-loss-pytorch - GitHub, <https://github.com/itakurah/focal-loss-pytorch>
 49. Implementing Focal Loss in PyTorch for Class Imbalance | by Amit Yadav - Medium, <https://medium.com/data-scientists-diary/implementing-focal-loss-in-pytorch-for-class-imbalance-24d8aa3b59d9>
 50. coral-pytorch - PyPI, <https://pypi.org/project/coral-pytorch/>
 51. Raschka-research-group/coral-pytorch: CORAL and CORN implementations for ordinal regression with deep neural networks. - GitHub, <https://github.com/Raschka-research-group/coral-pytorch>
 52. Deep Neural Networks for Rank-Consistent Ordinal Regression Based On Conditional Probabilities - arXiv, <https://arxiv.org/html/2111.08851v5>
 53. A Multilayer Perceptron for Ordinal Regression using CORAL -- Cement Dataset, https://raschka-research-group.github.io/coral-pytorch/tutorials/pytorch_lightning/ordinal-coral_cement/
 54. The Impact of Inconsistent Human Annotations on AI driven Clinical Decision Making, https://www.researchgate.net/publication/363599222_The_Impact_of_Inconsistent_Human_Annotations_on_AI_driven_Clinical_Decision_Making
 55. How to Calculate Cohen's Kappa in Python - Statology, <https://www.statology.org/cohens-kappa-python/>
 56. Do Androids Dream of Breaking the Game? Systematically Auditing AI Agent Benchmarks with BenchJack - arXiv, <https://arxiv.org/html/2605.12673v1>